

贪吃游戏软件分析与设计说明书

一、引言

1.1 项目背景

本项目基于 C 语言开发一款控制台贪吃蛇游戏，结合结构化分析、结构化设计、结对开发、版本控制完成全流程软件工程实践。项目包含用户注册、登录、游戏核心、日志记录、界面优化、算法优化、文件持久化等功能，旨在完整实践软件工程方法，提升结构化设计、模块化编码、团队协作能力。

1.2 项目目标

1. 完成贪吃蛇游戏需求建模、结构化设计、编码实现、测试交付全流程；
2. 实现用户注册、登录、游戏游玩、日志记录、数据持久化；
3. 采用高内聚、低耦合模块化设计；
4. 完成结对协作开发、GitHub 版本管理、文档交付。

1.3 文档范围

本文档涵盖：需求分析、数据设计、结构化设计、界面设计、算法设计、编码设计、测试设计、项目管理、交付成果。

二、软件需求分析

2.1 功能需求

2.1.1 用户管理

- 用户注册：用户名 3-20 位、密码 6-20 位，用户名唯一；
- 用户登录：读取 user.txt 校验账号密码；
- 用户信息持久化：user.txt 存储用户 ID、用户名、密码。

2.1.2 游戏核心功能

- 地图绘制、蛇移动、食物生成；
- 撞墙检测、自咬检测；
- 得分统计、游戏时长、速度控制；
- 方向控制：禁止反向掉头；
- 食物生成：不在墙、不在蛇身。

2.1.3 日志管理

- 游戏结束自动写入日志；
- 日志包含：日志 ID、用户 ID、用户名、开始时间、时长、得分；
- F5 按键查看当前用户日志；
- 日志持久化：log.txt 存储。

2.1.4 界面交互

- 主界面、注册界面、登录界面、游戏界面、结束界面、日志界面；

- 固定窗口：80 列 ×25 行；
- 左侧游戏区、右侧信息区；
- 实时显示用户名、得分、速度、操作提示。

2.2 非功能需求

- 易用：界面整洁、操作简单；
- 稳定：无死循环、无内存泄漏；
- 可维护：模块化、结构清晰；
- 可移植：标准 C 语言、主流编译器运行。

2.3 需求建模（实验三）

2.3.1 数据流图（DFD）

- 0 层 DFD：用户 → 系统 → user.txt、log.txt，如图 1 所示。
- 1 层 DFD（登录模块）：输入账号密码 → 合法性校验 → 登录成功 / 失败，如图 2 所示。

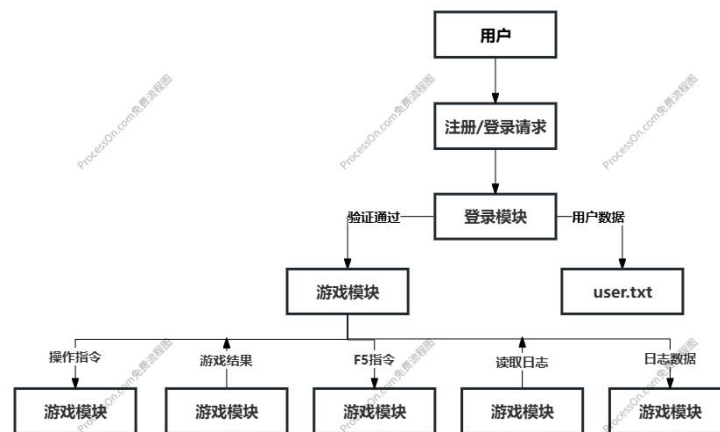


图 1 0 层数据流图

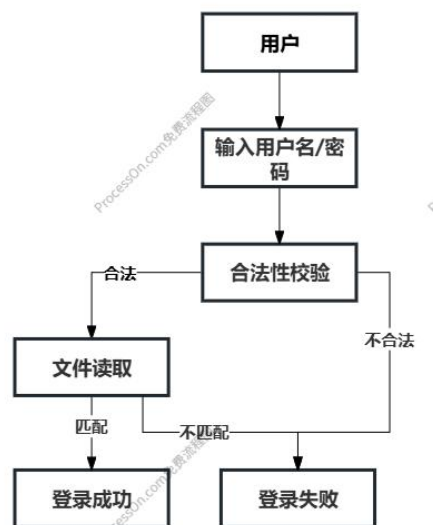


图 2 1 层数据流图

2.3.2 数据字典

数据项	类型	长度	说明
userId	整型	4 字节	用户唯一标识，自增主键
username	字符串	20 字节	登录用户名，3-20 位
password	字符串	20 字节	登录密码，6-20 位
logId	整型	4 字节	日志唯一标识，自增主键
starTime	字符串	50 字节	游戏开始时间，格式为系统时间
duration	整型	4 字节	游戏持续时长，单位为秒
score	整型	4 字节	本局游戏得分

2.3.3 E-R 图

- 实体：用户（User）：属性 id、username、password；
- 实体：游戏日志（GameLog）：属性 logId、userId、username、startTime、duration、score；
- 关系：一对多：一个用户对应多条日志。

如图 3 所示。

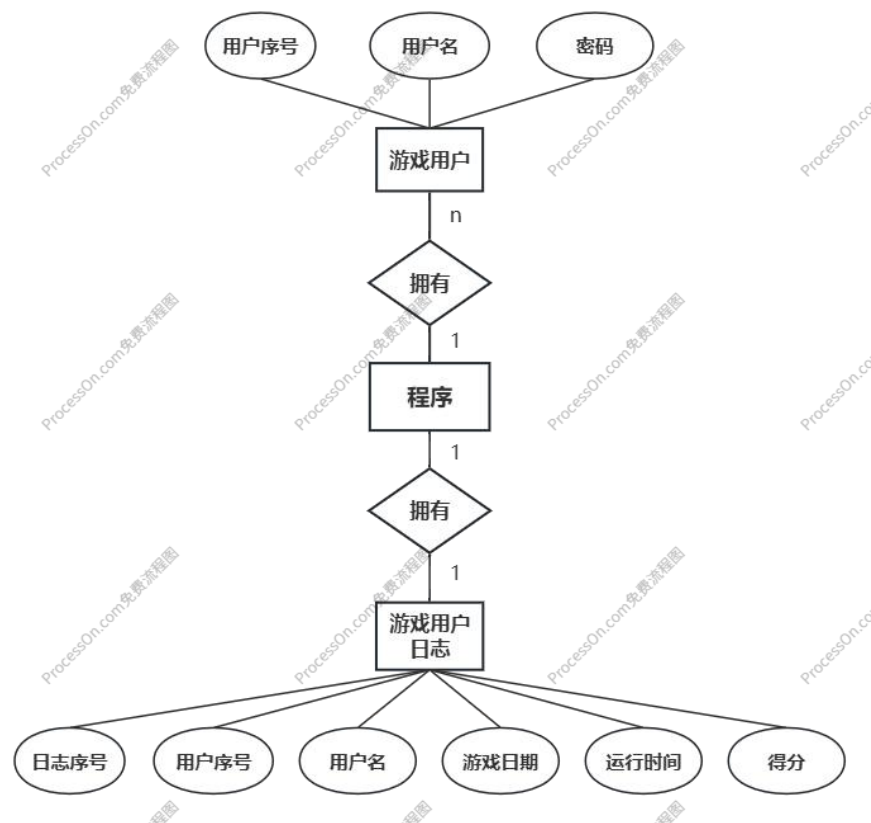


图 3 E-R 图

2.3.4 状态图

启动 → 主菜单 → 注册 / 登录 → 游戏中 → 游戏结束 → 查看日志 / 退出。
如图 4 所示。

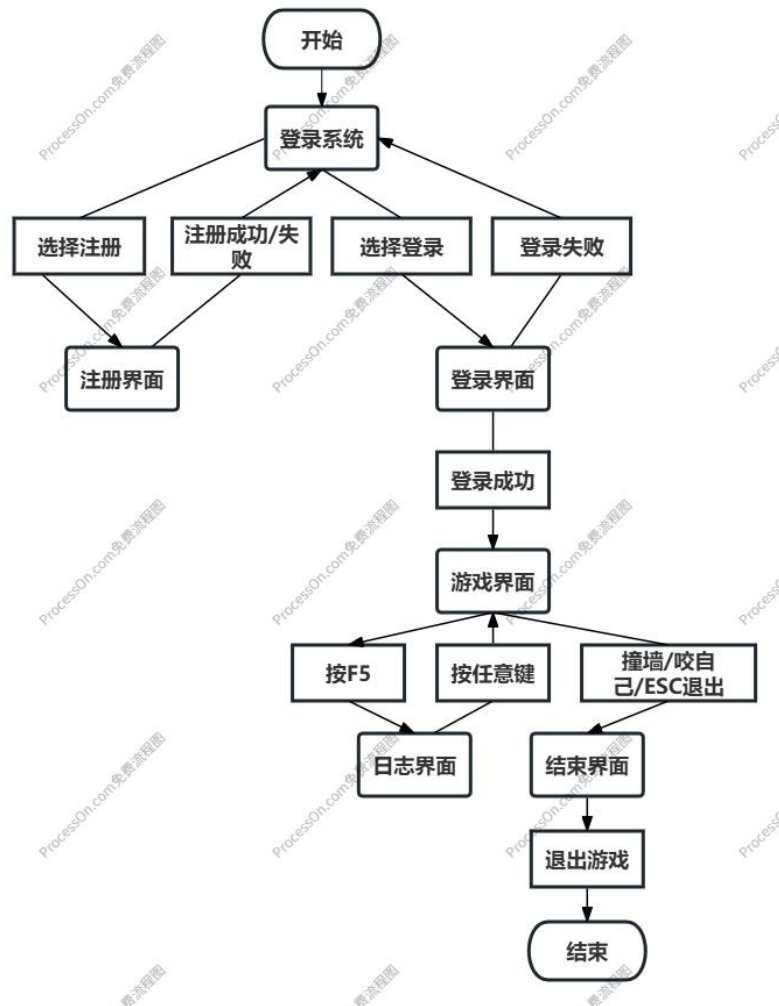


图 4 状态图

三、数据设计

3.1 结构体设计

3.1.1 用户结构体

```
typedef struct User {
    int id;           // 自增主键
    char username[20]; // 用户名
    char password[20]; // 密码
} User;
```

存储格式：用户 ID,用户名,密码

3.1.2 游戏日志结构体

结构体：

```
typedef struct GameLog {
```

```
int logId;           // 日志 ID
int userId;          // 关联用户 ID
char username[20];   // 用户名
char startTime[50];  // 开始时间
int duration;        // 时长(秒)
int score;           // 得分
} GameLog;
```

存储格式：日志 ID,用户 ID,用户名,开始时间,时长,得分、

3.2 文件存储设计

3.2.1 用户文件（user.txt）

格式： 用户 ID, 用户名, 密码

示例：

1,zhangsan,123456

2,lisi,654321

3.2.2 日志文件（log.txt）

格式： 日志 ID, 用户 ID, 用户名, 开始时间, 时长, 得分

示例：

1,1,zhangsan,2025-12-01 10:00:00,60,100

四、结构化设计

4.1 功能模块层次方框图

顶层：贪吃蛇游戏系统

- 用户管理模块：注册、登录、文件读写；
- 游戏核心模块：初始化、蛇移动、食物生成、碰撞检测、得分速度；
- 日志管理模块：日志记录、日志读取、日志显示；
- 界面模块：窗口控制、界面绘制、信息显示；
- 工具模块：随机数、时间、按键处理、数据校验。

设计原则：自上而下、分层拆分、高内聚、低耦合。

如图 5 所示

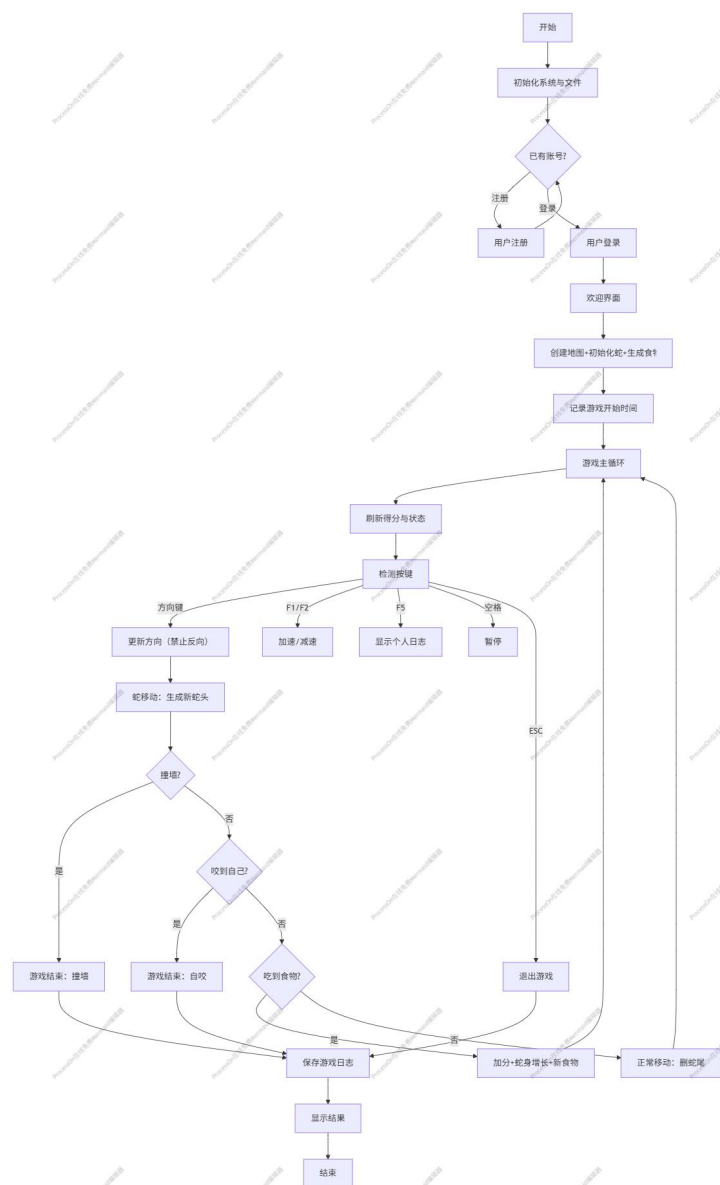


图 5 功能模块层次方框图

4.2 模块功能说明

1. 用户管理模块：用户注册、登录、合法性校验、文件读写；
2. 游戏核心模块：地图、蛇、食物、移动、碰撞、得分、速度；
3. 日志管理模块：记录、读取、展示游戏记录；
4. 界面模块：窗口、布局、绘制、刷新；
5. 工具模块：辅助功能。

五、界面设计

5.1 窗口布局

- 窗口大小：80 列 × 25 行；
- 左侧（60 列）：游戏区；
- 右侧（20 列）：信息区。

5.2 界面内容

- 主界面：标题、注册、登录、退出；
- 注册界面：用户名、密码输入及规则提示；
- 登录界面：账号密码输入、验证结果；
- 游戏界面：地图、蛇、食物、用户名、得分、速度；
- 结束界面：最终得分、时长、重玩、退出；
- 日志界面：用户历史游戏记录。

5.3 界面优化

- 统一排版、居中显示；
- 实时刷新、按键防抖；
- 界面整洁、交互友好。

六、算法设计与优化

6.1 核心算法

1. 蛇移动算法：链表头插法，高效移动；
2. 食物生成算法：随机生成、排除墙与蛇身；
3. 碰撞检测：先撞墙、后自咬；
4. 方向控制：禁止反向掉头；
5. 按键防抖：避免重复触发。

6.2 优化点

- 食物不在墙 / 蛇身上；
- 链表替代数组，提升效率；
- 逻辑清晰、减少无效判断；
- 提升稳定性与体验。

6.3 主流程

如图 6 所示。

启动 → 初始化 → 主菜单 → 登录 / 注册 → 游戏循环（绘制→输入→更新→检测）→ 结束 → 记录日志 → 返回 / 退出。

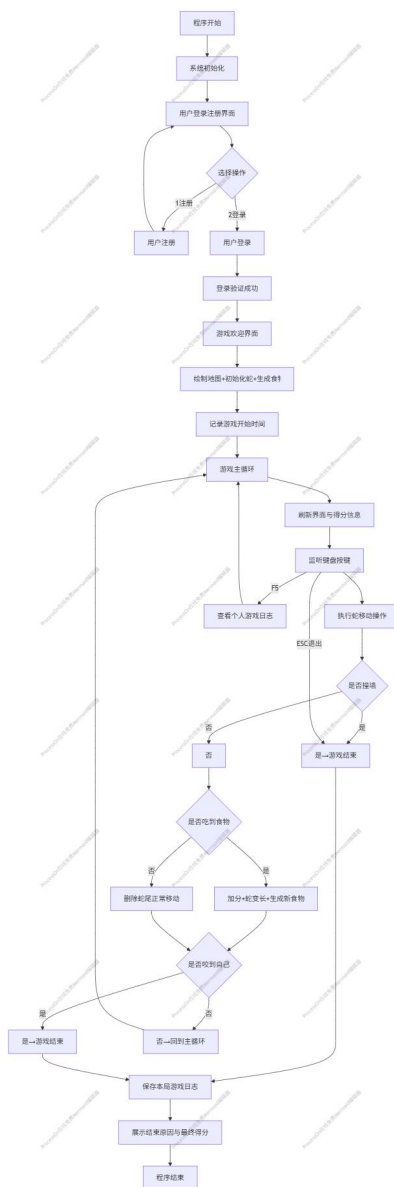


图 6 主流程图

七、编码设计

7.1 核心函数

- 用户函数：userRegister()、userLogin()、readUser()、writeUser()；
- 游戏函数：initGame()、moveSnake()、createFood()、checkCollision()、gameLoop()；
- 日志函数：saveLog()、showLog()、readLog()；
- 界面函数：drawUI()、drawGame()、showInfo()。

7.2 编码规范

- 模块化、函数分工明确；
- 注释清晰、命名规范；
- 结构清晰、便于维护。

八、测试设计

8.1 测试环境

- 系统：Windows;
- 编译器：Dev-C++ / Code::Blocks;
- 运行：控制台。

8.2 测试用例

- 注册：合法 / 非法用户名、密码、重复注册;
- 登录：正确 / 错误账号密码;
- 游戏：移动、食物、撞墙、自咬、得分、速度;
- 日志：记录、查看;
- 界面：显示、布局、刷新。

8.3 测试结果

所有功能正常、稳定、无严重 Bug，满足设计要求。

九、项目管理（WBS）

9.1 任务分解（WBS）

9.2 版本控制

项目托管至 GitHub，包含：

- 项目源码
- 软件分析与设计说明书

项目仓库链接：

十、总结

本次贪吃蛇游戏项目严格按照结构化软件工程方法完成分析、设计、编码、测试、交付全流程。结对协作分工明确、沟通高效，掌握 WBS 任务分解、结构化设计、模块化编码、文件持久化、版本控制能力。项目实现用户管理、游戏核心、日志记录、界面优化、算法优化，界面友好、代码规范、运行稳定，圆满完成实验目标，为后续软件开发打下坚实基础。